KỊCH BẢN VIDEO

KHÁNH — Backend API + Kết Luận Toàn Đề Án

XLA Face Privacy System — Đồ Án Cuối Kỳ

Em trực tiếp viết 2 files + đảm nhiệm phần kết luận toàn đề án:

`src/app_api.py`	FastAPI Server: 40+ endpoints, Pipeline, Streaming
`src/run_phone_cam.py`	FrameReader + Phone Camera Integration
Tổng cộng	

Đây là codebase lớn nhất trong nhóm. Khánh cũng là người trình bày cuối cùng — kết luận toàn bộ đề án.

Thời lượng mục tiêu: 14–16 phút

Cần chuẩn bị: Server `uvicorn` chạy cổng 8000 • VS Code mở `app_api.py` + `run_phone_cam.py` • Browser mở `dashboard.html` + `face-registry.html` + `secure-archive.html` • Swagger UI: `localhost:8000/docs` • Camera kết nối

⚠ CHÚ Ý

Lưu ý cho Khánh: `app_api.py` 1893 dòng — không thể show hết. Chiến lược: show 5–6 điểm quan trọng và giải thích tại sao chúng quan trọng. Phần kết luận toàn đề án: chuẩn bị số liệu cụ thể (749+759+3207 dòng = 4725 dòng, etc.)

Mục lục

1 CHUẨN BỊ TRƯỚC KHI BẤM REC **2**

2 0:00–0:15 — MỞ ĐẦU **3**

3 0:15–1:30 — GIỚI THIỆU ĐÓNG GÓP **3**

4 1:30–3:00 — APP_API.PY — ADMIN AUTHENTICATION **3**

5 3:00–6:00 — PIPELINESESSION CLASS — LIVE VIDEO PIPELINE **5**

5.1 3:00–4:30 — PipelineSession __init__, start, stop, patch_config 5

5.2 4:30–6:00 — _run: processing loop (dòng 636–750) 6

6 6:00–7:30 — RUN_PHONE_CAM.PY — FRAMEREADER **7**

7 7:30–9:30 — STREAMING ENDPOINTS **9**

7.1 7:30–8:15 — MJPEG Stream (dòng 1540–1572) 9

7.2 8:15–9:30 — SSE và WebSocket (dòng 1581, 1620) 10

8 9:30–11:00 — CRUD ENDPOINTS (bảng tổng hợp) **10**

9 11:00–12:30 — DEMO LIVE — FRONTEND TOUR **13**

10 12:30–15:00 — KẾT LUẬN TOÀN ĐỀ ÁN **14**

11 PHỤ LỤC — CÂU HỎI DỰ KIẾN CHO KHÁNH **15**

1. CHUẨN BỊ TRƯỚC KHI BẤM REC

THAO TÁC

Bước 1 — Khởi động server:

```
cd C:\Users\itent\Desktop\xla_demo
.venv\Scripts\activate
uvicorn src.app_api:app --host 0.0.0.0 --port 8000 --reload
```

Chờ thấy: `Application startup complete` trên terminal.

THAO TÁC

Bước 2 — VS Code: Tab 1: `src/app_api.py` (1893 dòng)
Tab 2: `src/run_phone_cam.py` (1314 dòng)
Font size 16–18.

THAO TÁC

Bước 3 — Browser (3 tab): Tab 1: `http://localhost:8000/admin/dashboard.html`
Tab 2: `http://localhost:8000/admin/face-registry.html`
Tab 3: `http://localhost:8000/admin/secure-archive.html`
Tab 4 (để Q&A): `http://localhost:8000/docs` (Swagger auto-generated)

CAMERA

Camera ngang tầm mắt. Ánh sáng từ phía trước. Nói rõ ràng, xương sống thẳng. Video này kết thúc toàn bộ đề án — tone tự tin, không vội vàng.

2. 0:00–0:15 — MỞ ĐẦU

0:00 – 0:15

Nhìn camera, mở đầu.

NÓI

"Xin chào thầy cô. Em là Khánh.
Em phụ trách phần **Backend API và hệ thống Pipeline xử lý video real-time**. Video này em trình bày 2 files em viết, sau đó kết luận toàn bộ đề án của nhóm."

3. 0:15–1:30 — GIỚI THIỆU ĐÓNG GÓP

THAO TÁC

Hover chuột qua từng tab file khi nói.

NÓI

"Em viết 2 files lớn nhất trong đề án:
app_api.py — 1893 dòng. Đây là trung tâm của toàn hệ thống — FastAPI server. 40+ HTTP endpoints, WebSocket, MJPEG stream, pipeline orchestration. Mọi thứ Đạt và Khoa viết đều được gọi thông qua server này.
run_phone_cam.py — 1314 dòng. Trong file này có class `FrameReader` — thread đọc camera non-blocking. Cũng có pipeline xử lý khi chạy từ command line.
Tổng cộng 3207 dòng — nhiều nhất nhóm.
Em sẽ chỉ rõ 6 điểm quan trọng nhất: Admin authentication, PipelineSession class, FrameReader, main processing loop, streaming, và CRUD endpoints."

4. 1:30–3:00 — APP_API.PY — ADMIN AUTHENTICATION

1:30 – 3:00

Chuyển sang `src/app_api.py`.

THAO TÁC

Click tab `app_api.py`. Nhấn `Ctrl+G`, gõ **394**, Enter.

NÓI

"Điểm đầu tiên: **Admin Authentication flow**.
`app_api.py` không tự xử lý password — nó delegate hết cho `admin_auth.py` của Đạt. Nhưng

em viết phần JWT token và middleware protect endpoints.”

CODE — app_api.py dòng 394–448 — Admin auth: token + middleware

```
394 def _issue_admin_token() -> str:           # dong 394
395     """Issue a short-lived admin JWT (no library)."""
396     payload = {
397         "sub": "admin",
398         "iat": int(time.time()),
399         "exp": int(time.time()) + 3600,    # 1 hour
400     }
401     token_b = json.dumps(payload).encode()
402     return base64.urlsafe_b64encode(token_b).decode()
403
404 def _verify_admin_token(token: str) -> bool:    # dong 408
405     try:
406         payload = json.loads(base64.urlsafe_b64decode(token))
407         return (payload.get("sub") == "admin" and
408                 time.time() < payload.get("exp", 0))
409     except Exception:
410         return False
411
412 async def _require_admin_auth(           # dong 426: middleware
413     request: Request,
414     x_admin_token: Optional[str] =
415         Header(None, alias="X-Admin-Token"),
416 ) -> None:
417     if x_admin_token and _verify_admin_token(x_admin_token):
418         return
419     # Also check cookie
420     cookie_tok = request.cookies.get("admin_token", "")
421     if cookie_tok and _verify_admin_token(cookie_tok):
422         return
423     raise HTTPException(status_code=401,
424                         detail="Admin authentication required")
```

NÓI

“**Dòng 394**: `_issue_admin_token` tạo token JSON base64. Token chứa: sub="admin", thời điểm tạo, thời điểm hết hạn (1 tiếng).
Dòng 408: `_verify_admin_token` decode base64, parse JSON, verify sub="admin"và chưa expired.
Dòng 426: `_require_admin_auth` là dependency FastAPI. Mọi endpoint cần quyền admin đều khai báo `Depends(_require_admin_auth)`. Kiểm tra header `X-Admin-Token` hoặc cookie `admin_token`. Nếu cả hai đều fail: HTTP 401.
Điểm quan trọng: chỉ sau khi Đạt’s `verify_admin` trong `admin_auth.py` trả True em mới gọi `_issue_admin_token` ở dòng 394. Không có password đúng → không có token → không qua middleware dòng 426.”

5. 3:00–6:00 — PIPELINESESSION CLASS — LIVE VIDEO PIPELINE

3:00 – 6:00

Nhấn `Ctrl+G`, gõ **512**, Enter.

NÓI

"Điểm thứ hai và quan trọng nhất: class `PipelineSession`."

5.1 3:00–4:30 — PipelineSession __init__, start, stop, patch_config

CODE — app_api.py dòng 512–590 — PipelineSession: pipeline lifecycle management

```
512 class PipelineSession:
513     """
514     Manages one live video pipeline: camera -> detect -> anonymize ->
515     stream. One instance per active pipeline.
516     """
517
518     def __init__(self, camera_index: int,
519                  preset_name: str = "balanced") -> None:  # dong 515
520         self.camera_index = camera_index
521         self.preset_name  = preset_name
522         self.stop_event   = threading.Event()
523         self.cfg_lock     = threading.Lock()
524         self.live_cfg     = _LiveConfig(preset_name=preset_name)
525         self._thread: Optional[threading.Thread] = None
526         self.frame_buffer = _FrameBuffer()        # output buffer
527
528     def start(self) -> None:                      # dong 571
529         self.stop_event.clear()
530         self._thread = threading.Thread(
531             target=self._run, daemon=True)
532         self._thread.start()
533
534     def stop(self) -> None:                       # dong 575
535         self.stop_event.set()                     # signal stop
536         if self._thread:
537             self._thread.join(timeout=10.0)       # wait graceful shutdown
538
539     def is_alive(self) -> bool:                   # dong 580
540         return (self._thread is not None and
541                 self._thread.is_alive())
542
543     def patch_config(self,                        # dong 583
544                      patch: Dict[str, Any]) -> None:
545         with self.cfg_lock:                       # dong 585: atomic
546             for key, value in patch.items():
547                 setattr(self.live_cfg, key, value)# dong 587: hot-reload
```

NÓI

"**Dòng 515**: `__init__` nhận camera index và preset name. `stop_event` là threading.Event — cờ hiệu để dừng pipeline. `cfg_lock` là Lock để protect live config khi hot-reload.
Dòng 571: `start()` tạo thread daemon, gọi `self._run()`. Daemon thread: tự chết khi main process chết — không leak process.
Dòng 575: `stop()` — hai bước quan trọng: Dòng 577: `stop_event.set()` — set cờ. Thread `_run()` kiểm tra cờ này ở cuối mỗi vòng loop và tự thoát. Dòng 578: `join(timeout=10.0)` — chờ tối đa 10 giây để thread thoát gracefully. Timeout để không block mãi mãi nếu thread bị stuck.
Dòng 583–587: `patch_config` — hot-reload config không restart pipeline. Client frontend gửi PATCH /pipeline/config. `with self.cfg_lock`: đảm bảo atomic — thread pipeline không đọc config lúc đang write. `setattr`: thay đổi key-value bất kỳ trong `live_cfg`. Ví dụ: đổi từ balanced sang strong, hoặc đổi detect_every từ 2 sang 1. Pipeline đang chạy nhận config mới ngay frame tiếp theo — không cần restart."

5.2 4:30–6:00 — _run: processing loop (dòng 636–750)

THAO TÁC

Nhấn `Ctrl+G`, gõ **636**, Enter.

CODE — app_api.py dòng 636–752 — _run(): main processing loop

```
636 def _run(self) -> None:
637     """Main pipeline loop running in background thread."""
638     cam_idx  = self.camera_index
639     cap      = cv2.VideoCapture(cam_idx)      # dong 639
640
641     # --- Khoi dong 3 module tu cac file khac ---
642     detector   = FaceDetector(                # dong 674: tu Khoa
643         model_path=MODEL_PATH,
644         conf_threshold=cfg.conf_threshold,
645         iou_threshold=cfg.iou_threshold,
646         imgsz=cfg.imgsz)
647     db         = FamilyDatabase(              # dong 688: tu Khoa
648         db_path=str(_P("data/family_members.json")))
649     recognizer = FaceRecognizer(              # dong 692: tu Khoa
650         model=cfg.recognition_model)
651
652     frame_reader = FrameReader(cap)           # dong 711: tu run_phone_cam.
        py
653     frame_reader.start()                      # dong 712: thread start
654
655     frame_idx = 0
656
657     while not self.stop_event.is_set():       # dong 738: loop cho den khi
        stop
658         ok, frame = frame_reader.read()       # dong 739: non-blocking read
659         if not ok:
660             time.sleep(0.04)
```

```
            continue
        frame_idx += 1

        if frame_idx % cfg.detect_every == 0:  # dong 750: YOLO every N frames
            boxes_scores = detector.detect_faces_with_scores(frame)
```

NÓI

"**Dòng 639**: `cv2.VideoCapture(cam_idx)` mở kết nối camera.
Dòng 674: `FaceDetector` của Khoa. Khởi động một lần, tái dùng mãi mãi. **Dòng 688**: `FamilyDatabase` của Khoa. Load JSON từ disk vào RAM. **Dòng 692**: `FaceRecognizer` của Khoa. Load InsightFace buffalo_l model.
Ba module này **em không viết** — em chỉ orchestrate chúng ở đây. Đây là Separation of Concerns: Khoa viết class riêng, em gọi từ pipeline.
Dòng 711–712: `FrameReader(cap)` và `start()` — khởi động thread đọc frame. `FrameReader` em giải thích ngay sau.
Dòng 738: `while not self.stop_event.is_set()` — loop tới khi `stop()` gọi.
Dòng 750: `frame_idx % cfg.detect_every == 0` — chỉ chạy YOLO mỗi N frame. `detect_every=2` (preset balanced): YOLO chạy 15fps, display vẫn 30fps. Tiết kiệm 50% CPU cho YOLO — box cũ vẫn apply cho frame bỏ qua."

6. 6:00–7:30 — RUN_PHONE_CAM.PY — FRAMEREADER

6:00 – 7:30

Chuyển sang tab `src/run_phone_cam.py`.

THAO TÁC

Click tab `run_phone_cam.py`. Nhấn `Ctrl+G`, gõ **327**, Enter.

NÓI

"Điểm thứ ba: `FrameReader` class — tại sao cần một thread riêng để đọc camera?"

CODE — run_phone_cam.py dòng 327–382 — FrameReader: non-blocking camera reader

```
327 class FrameReader(threading.Thread):
328     """
329     Dedicated thread to drain frames from VideoCapture.
330     Keeps queue size = 1 so pipeline always gets THE LATEST frame,
331     not a stale one from 500ms ago.
332     """
333
334     def __init__(self, cap: cv2.VideoCapture) -> None:
```

```
331         super().__init__(daemon=True)              # dong 331
332         self.cap     = cap
333         self.queue: Queue = Queue(maxsize=1)       # dong 332: chi 1 slot
334         self.running = True
335
336     def run(self) -> None:                         # dong 338
337         while self.running:
338             ok, frame = self.cap.read()            # dong 341: blocking
339             if not ok:
340                 time.sleep(0.02)
341                 continue
342             # Drop-stale pattern: loai bo frame cu neu queue day
343             if self.queue.full():                  # dong 365
344                 try:
345                     self.queue.get_nowait()        # dong 366:  b   frame
                           cu
346                 except Empty:
347                     pass
348             try:
349                 self.queue.put_nowait(frame)       # dong 369: push moi
                       nhat
350             except Full:
351                 pass
352
353     def read(self, timeout: float = 0.6):          # dong 376
354         try:
355             return True, self.queue.get(           # dong 378
356                 timeout=timeout)
357         except Empty:
358             return False, None                     # dong 380
359
360     def stop(self) -> None:                        # dong 382
361         self.running = False
```

NÓI

"**Vấn đề nếu không có FrameReader:** `cv2.VideoCapture.read()` là blocking call — nó block thread cho tới khi camera có frame. Nếu pipeline đang xử lý (YOLO + anonymize) mất 80ms, `cap.read()` trong cùng thread phải chờ. Trong 80ms đó camera có 2–3 frame mới — tất cả bị drop trong OpenCV buffer. Khi `cap.read()` tiếp theo: trả về frame từ 80ms trước — **stale**. Stream nhìn như lag.

Giải pháp — FrameReader: Dòng 327: inherit `threading.Thread`. **Dòng 331**: `daemon=True` — tự chết khi process chết. **Dòng 332**: `Queue(maxsize=1)` — chỉ 1 slot. Không phải 30 slot (framerate buffer) — chỉ 1.

Dòng 341: `cap.read()` chạy trong thread riêng. Nó block — nhưng không ảnh hưởng pipeline thread.

Dòng 365–369: đây là **drop-stale pattern**: Nếu queue đã đầy (có frame chưa được lấy): Bỏ frame cũ đó đi (dòng 366). Đẩy frame mới vào (dòng 369). Kết quả: queue luôn chứa frame **mới nhất**, không phải frame cũ.

Dòng 376: pipeline gọi `frame_reader.read(timeout=0.6)`. Nếu trong 0.6s không có frame (camera disconnect): trả False. Không block pipeline mãi mãi."

7. 7:30–9:30 — STREAMING ENDPOINTS

7:30 – 9:30

Quay về tab `app_api.py`.

THAO TÁC

Nhấn `Ctrl+G`, gõ **1540**, Enter.

NÓI

"Điểm thứ tư: làm sao browser nhận video real-time từ server? Ba protocol: MJPEG, SSE, WebSocket."

7.1 7:30–8:15 — MJPEG Stream (dòng 1540–1572)

CODE — app_api.py dòng 1540–1572 — GET /stream/mjpeg: MJPEG multipart stream

```
1540 @app.get("/stream/mjpeg")
1541 async def stream_mjpeg(
1542     session_id: Optional[str] = None,
1543     ...
1544 ) -> StreamingResponse:
1545
1546     async def generate():
1547         while True:
1548             frame_data = await asyncio.get_event_loop(
1549             ).run_in_executor(None,
1550                 session.frame_buffer.get, 0.5)  # dong 1553: get frame
1551
1552             if frame_data is None:
1553                 await asyncio.sleep(0.04)        # dong 1555: 25fps cap
1554                 continue
1555
1556             # MJPEG boundary format: RFC 2046 multipart
1557             yield (b"--frame\r\n"                # dong 1560: boundary
1558                    b"Content-Type: image/jpeg\r\n\r\n"
1559                    + frame_data + b"\r\n")
1560             await asyncio.sleep(0.04)            # dong 1563: 25fps
1561
1562     return StreamingResponse(
1563         generate(),
1564         media_type="multipart/x-mixed-replace"  # dong 1567
1565                    ";boundary=frame",
1566     )
```

NÓI

"MJPEG là cách đơn giản nhất để stream video qua HTTP.
Dòng 1560: `-frame\r\n` là boundary separator. Server gửi sequence: [boundary][header][jpeg bytes][boundary][header][jpeg bytes]... Browser <img> tag decode từng JPEG liên tiếp — thành video.
Dòng 1555, 1563: `asyncio.sleep(0.04)` = 25fps cap. Tại sao 25fps chứ không 30? Camera 30fps, pipeline xử lý ≈25fps. Cap ở 25 tránh buffer overflow.
Tại sao không WebSocket luôn? MJPEG với `<img>` tag không cần JS — works với `<img src="/stream/mjpeg">`. Lowest latency setup. WebSocket cần JS to rebuild frames."

7.2 8:15–9:30 — SSE và WebSocket (dòng 1581, 1620)

THAO TÁC

Nhấn `Ctrl+G`, gõ **1581**, Enter. Đọc 5 giây. Nhấn `Ctrl+G`, gõ **1620**, Enter.

NÓI

"**Dòng 1581**: `GET /stream/sse` — Server-Sent Events. Thay vì stream JPEG bytes, stream JSON metadata: bao nhiêu khuôn mặt, tên ai đã nhận ra, thống kê. Frontend dùng để update số liệu dashboard mà không cần reload.
Dòng 1620: `@app.websocket("/stream/ws")` — WebSocket. Dùng khi client muốn bidirectional: subscribe stream và đồng thời gửi lệnh. Ví dụ: đang stream, client gửi `{"action":"unlock","member_id":"dat"}`. Server nhận, unlock track đó ngay, stream tiếp.
Ba protocol cover mọi use case: MJPEG cho display đơn giản, SSE cho metadata, WebSocket cho control."

8. 9:30–11:00 — CRUD ENDPOINTS (bảng tổng hợp)

9:30 – 11:00

Mở `localhost:8000/docs` trên browser — Swagger UI.

NÓI

"Điểm thứ năm: tổ chức endpoint.
Em gom 40+ endpoint thành groups rõ ràng. Mở Swagger để thấy full list."

THAO TÁC

Chuyển sang browser tab Swagger UI (`localhost:8000/docs`). Scroll qua các group endpoint.

Tổng hợp endpoints chính (trích dẫn dòng)

Method	Endpoint	Dòng (app_api.py)	Mục đích
GET	/health	968	Kiểm tra server alive
GET	/presets	988	Trả PRESETS dict của Đạt
POST	/anonymize	1010	Ẩn danh 1 ảnh tĩnh (no pipeline)
POST	/lock	1035	Lock track khỏi whitelist
POST	/unlock	1074	Unlock track về whitelist
POST	/recognize	1102	Nhận dạng 1 ảnh tĩnh
GET	/members	1261	Danh sách thành viên đã đăng ký
POST	/members	1305	Đăng ký thành viên mới
PATCH	/members/{id}	1375	Cập nhật thông tin / thêm embedding
DELETE	/members/{id}	1408	Xoá thành viên
POST	/pipeline/start	1432	Khởi động PipelineSession (dòng 512)
DELETE	/pipeline/stop	1458	Dừng pipeline, join thread (dòng 575)
GET	/pipeline/status	1474	Trạng thái: alive/stopped, frame count
PATCH	/pipeline/config	1488	Hot-reload config (gọi patch_config dòng 583)
GET	/stream/mjpeg	1540	MJPEG stream (25fps)
GET	/stream/sse	1581	SSE metadata stream
WS	/stream/ws	1620	WebSocket bidirectional
POST	/admin/setup	1707	Tạo admin lần đầu (delegate sang Đạt auth)
POST	/admin/verify	1725	Login admin, issue JWT token
GET	/clips	1763	Danh sách clips đã ghi
GET	/clips/{id}/video	1792	Download clip (raw, không decrypt)
POST	/clips/{id}/decrypt	1835	Decrypt clip với password
DELETE	/clips/{id}	1876	Xoá một clip
DELETE	/clips	1884	Xoá tất cả clips

NÓI

"Mỗi endpoint em sẽ chỉ một ví dụ quan trọng: **PATCH /pipeline/config**."

THAO TÁC

Quay về VS Code, `Ctrl+G`, gõ **1488**, Enter.

CODE — app_api.py dòng 1488–1504 — PATCH /pipeline/config: hot-reload runtime

```
1488 @app.patch("/pipeline/config",
1489            dependencies=[Depends(_require_admin_auth)])  # dong 1489: auth
1490 async def patch_pipeline_config(
1491     patch: Dict[str, Any],
1492     session_id: Optional[str] = None,
1493 ) -> dict:
1494     session = _get_session(session_id)
1495     if session is None:
1496         raise HTTPException(404, "No active pipeline")
1497
1498     allowed = {"preset_name", "detect_every",     # dong 1498: whitelist
1499                "blur_scale", "pixel_block",
1500                "head_ratio", "anonymize"}
1501     for key in patch:
1502         if key not in allowed:
1503             raise HTTPException(400, f"Unknown key: {key}")
1504     session.patch_config(patch)                   # dong 1504: atomic
         update
1505     return {"status": "ok", "applied": patch}
```

NÓI

"**Dòng 1489**: `Depends(_require_admin_auth)` — admin only. **Dòng 1498**: whitelist các keys được phép patch. Tránh client truyền arbitrary attributes vào config object — injection prevention. **Dòng 1504**: gọi `session.patch_config(patch)` — atomic update dòng 583. Response trả lại chính xác những gì đã apply — cho client confirm."

9. 11:00–12:30 — DEMO LIVE — FRONTEND TOUR

11:00 – 12:30

Chứng minh toàn bộ stack hoạt động end-to-end.

THAO TÁC

Mở browser tab 1: `http://localhost:8000/admin/dashboard.html`.

NÓI

"Demo nhanh 3 trang."

THAO TÁC

Demo 1 — Dashboard (dòng 1432, 1488, 1540):
1. Click "Start Pipeline" → POST /pipeline/start dòng 1432 → PipelineSession.start() dòng 571.
2. Video stream hiện ra → MJPEG từ /stream/mjpeg dòng 1540.
3. Dropdown đổi preset "strong" → PATCH /pipeline/config dòng 1488 → patch_config dòng 583.
4. Thấy blur mode thay đổi ngay không restart.
Nói: "Hot-reload, không cần restart pipeline."

THAO TÁC

Demo 2 — Face Registry (dòng 1305, 1261):
Chuyển sang tab face-registry.html.
Thấy danh sách thành viên → GET /members dòng 1261.
Nói: "Đây là database của Khoa's family_database.py được expose qua endpoint của em."

THAO TÁC

Demo 3 — Secure Archive (dòng 1763, 1835):
Chuyển sang tab secure-archive.html.
Danh sách encrypted clips → GET /clips dòng 1763.
Bấm decrypt trên một clip, nhập password → POST /clips/{id}/decrypt dòng 1835.
Nói: "Clip được encrypt bởi Đạt's clip_recorder.py AES-256-GCM. Endpoint này gọi decrypt_clip() của Đạt."

10. 12:30–15:00 — KẾT LUẬN TOÀN ĐỀ ÁN

12:30 – 15:00

QUAN TRỌNG NHẤT. Nhìn camera. Nói chậm, rõ. Đây là phần kết thúc toàn bộ đề án.

NÓI

"Em là người trình bày cuối — vậy em xin kết luận toàn bộ đề án.
Ba thành viên, ba lớp trong pipeline:
Đạt viết **lớp Privacy và Security**: `privacy.py` 8 thuật toán ẩn danh, `anonymizer_backend.py` 4 preset cấu hình, `clip_recorder.py` AES-256-GCM encrypted storage, `admin_auth.py` PBKDF2 390.000 iterations + timing-safe compare. Tổng 749 dòng.
Khoa viết **lớp Detection và Recognition**: `detector.py` YOLOv11n-face,

`face_recognition.py` InsightFace buffalo_l embedding 512-dim, `family_database.py` vectorized cosine matching, `bbox_smoother.py` EMA tracking. Tổng 759 dòng.
Em viết **lớp Orchestration và API**: `app_api.py` 40+ endpoints, PipelineSession, JWT middleware, 3 streaming protocol, hot-reload config, `run_phone_cam.py` FrameReader non-blocking drop-stale pattern. Tổng 3207 dòng.
Tổng toàn đề án: $749 + 759 + 3207 = 4715$ **dòng code.**
[dừng 1 giây] [nhìn camera]
Ba quyết định thiết kế quan trọng nhất:
Thứ nhất: **Privacy by Design**. Không có option "nhận diện trước, quyết định blur sau". Mọi khuôn mặt detect được đều bị xử lý ngay. Whitelist (thành viên gia đình) là exception, không phải default.
Thứ hai: **Separation of Concerns**. Privacy logic ở `privacy.py`. Detection ở `detector.py`. Recognition ở `family_database.py`. Orchestration ở `app_api.py`. Mỗi file có thể test, thay thế, nâng cấp độc lập. Ví dụ: thay YOLOv11 bằng YOLOv12 → chỉ sửa `detector.py`, pipeline không đổi.
Thứ ba: **Security theo chuẩn OWASP**. Không hardcode credentials. PBKDF2 390.000 iterations chống brute-force. AES-256-GCM với salt ngẫu nhiên mỗi clip. Timing-safe compare chống timing attack. JWT check trên mọi admin endpoint.
[dừng 1 giây] [nhìn camera]
Hạn chế và hướng phát triển:
Hạn chế: buffalo_l model 500MB — cold start chậm. InsightFace chạy CPU: 50–80ms/inference, nếu nhiều người trong frame đồng thời có thể lag. Chưa có caching embedding thực sự nhanh (FAISS, Annoy).
Hướng phát triển: Thay InsightFace bằng model nhỏ hơn cho edge device. Thêm FAISS index cho database lớn hơn. Certificate-based auth thay JWT base64 đơn giản.
[dừng 1 giây] [nhìn camera]
Đây là toàn bộ đề án XLA Face Privacy System. Cảm ơn thầy cô đã theo dõi."

11. PHỤ LỤC — CÂU HỎI DỰ KIẾN CHO KHÁNH

Q: Tại sao FastAPI chứ không Flask hay Django?

"FastAPI: async-native — quan trọng cho streaming (MJPEG, SSE, WebSocket) cùng lúc nhiều client. Flask: sync mặc định, streaming phải dùng monkey-patch. Django: too heavy cho task này, ORM không cần thiết. FastAPI auto-generate Swagger UI từ Python type hints — zero effort documentation."

Q: JWT token base64 không phải signed — có an toàn không?

"Đúng — đây là điểm hạn chế của implementation hiện tại. Token chỉ base64 encode, không sign bằng secret key. Ai decode được token có thể forge token hết hạn → đặt exp tương lai. Trong phạm vi đề án nội bộ (LAN, không expose internet) chấp nhận được. Production: dùng PyJWT với HS256 secret, thêm refresh token."

Q: detect_every=2 có làm miss khuôn mặt không?

"Không miss — YOLO chạy lần 2, box kết quả apply cho cả frame 1 và frame 2. Frame lẻ dùng box từ lần detect trước kết hợp BBoxSmoother của Khoa. Người di chuyển ở 30fps: 2 frame = 66ms. Ở 66ms người di chuyển ≈3–5cm — box prediction vẫn valid. Nếu cần accuracy cao hơn: set detect_every=1 qua PATCH /pipeline/config."

Q: FrameReader Queue(maxsize=1) có bị mất frame clip không?

"FrameReader phục vụ **live display** — chỉ cần frame mới nhất. ClipRecorder (của Đạt, dòng 248) có queue riêng, maxsize=300 (10s x 30fps). Two separate queues: FrameReader queue for display (latest), ClipRecorder queue for recording (all frames). Không dùng chung queue → không conflict."

Q: Làm sao chạy thực tế? Cần GPU không?

"Không cần GPU. YOLOv11n-face trên CPU Intel i5 gen 10: ≈25–30ms/frame. InsightFace buffalo_l CPU: ≈50–80ms/frame. Tổng pipeline latency: 80–120ms end-to-end. Display lag ≈0.1s — mắt thường không nhận ra. Máy yếu hơn: tăng detect_every lên 3–4, giảm imgsz xuống 416."